

03 Prediction as Learning

BY NILUSHANAN KULASINGHAM

12 MIN READ · INTERACTIVE

Ask something to predict what comes next and it has no choice but to build whatever the next moment depends on. The same loop, read a second way, turns out to be compression.

Here is a photograph of a ball in mid-air. Where is it a moment later?

You cannot say. Up, down, sideways, or barely moving at all: the picture looks the same in every case. Everything you would need to answer is missing from it.

Now take two photographs, a fraction of a second apart. The answer arrives before you have finished asking the question.

What changed is not the amount of detail. Both pictures were equally sharp. What changed is that two frames hold something one frame cannot: a direction and a speed. Nobody drew those on the image. They are not in either picture. They exist only in the relationship between them, and you pulled them out without being told to.

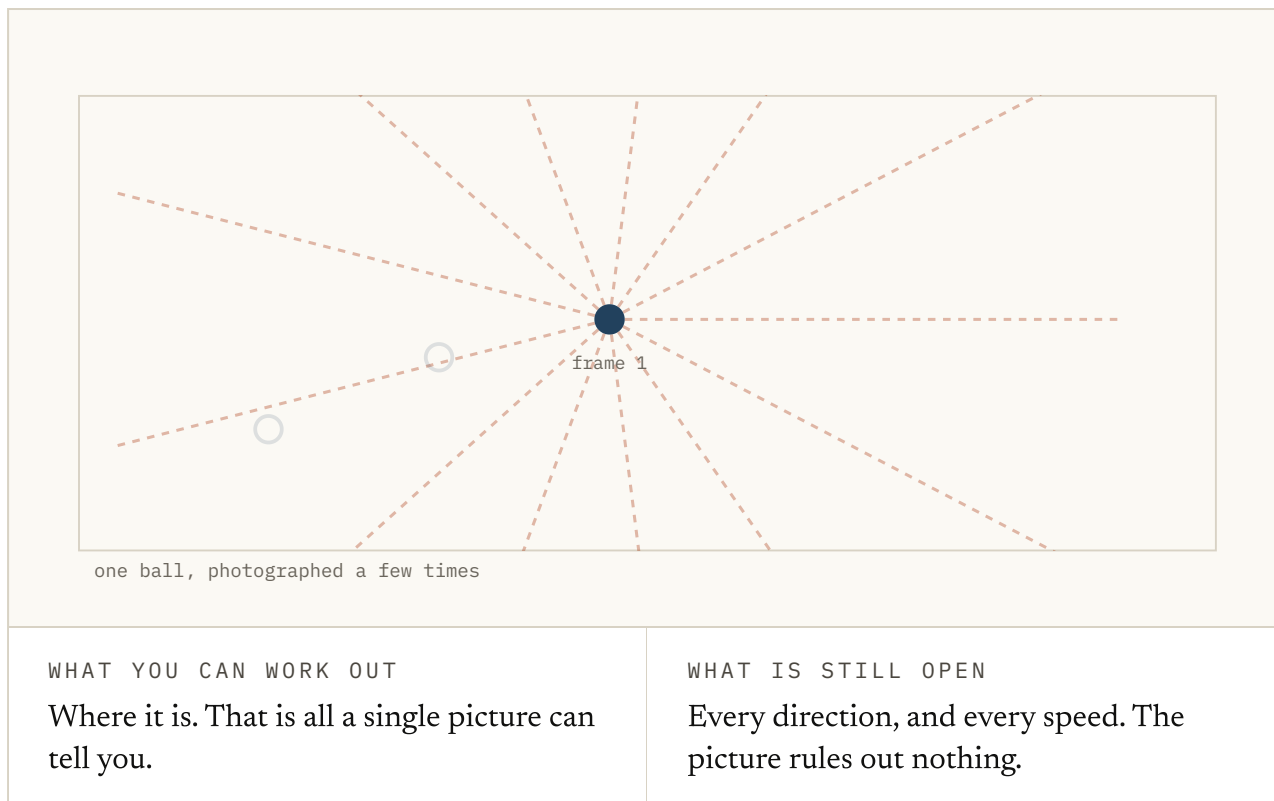


FIG. 3.1 The fan is every future still consistent with what has been seen. One frame rules out nothing. A second fixes the heading and the speed. A third settles whether the path is bending. Each extra frame is not more detail, it is one more thing you can work out that was never in any single picture.

That is the trick this chapter is about. Ask something to predict what comes next, and it has no choice but to build the things the next moment depends on. Not because you asked for them. Because it cannot be right without them.

Guess, check, adjust

The loop is as small as it sounds.

Look at what you have. Say what you think is coming. Wait. Compare what arrived to what you said. Change yourself a little in the direction that would have been less wrong. Do it again.

What makes it powerful is not the cleverness of any step. It is that the right answer is already there. Nobody has to label anything, because the thing you are trying to predict is the very next piece of the recording, and the recording already contains it.

This is what *self-supervised* means. Supervised, in the sense that the model is told exactly how wrong it was. But told by the data itself, rather than by a person with a spreadsheet.

That sounds like a small administrative convenience. It is the whole reason this approach ate the field. Labelling is the expensive part of machine learning: somebody has to look at a million images and write down what is in them. Prediction skips it. Any recording of anything is already a training set, because every moment in it is the answer to the moment before.



FIG. 3.2 A predictor guessing the next number from the previous two, corrected a little after every guess. It starts at zero and knows nothing. Watch the dashed line climb onto the solid one, then look at what it learned: two numbers that nobody supplied, arrived at only by being wrong and adjusting.

Nothing in that demonstration was told the rule. It was told, two hundred and twenty times, by how much it had just missed. Each time it nudged its own two numbers (the *weights*: the adjustable numbers inside a model, and the only thing that

learning ever changes) a little way towards being right. That was enough.

What guessing forces you to build

Here is where it stops being a party trick.

To predict well, you have to carry whatever the next moment depends on. If the sequence is a ball, that is position and velocity. If the sequence is language, it is a great deal more, and none of it was ever asked for directly.

In 1990 Jeffrey Elman trained a small network on one job: read a word, predict the next word. Then he went looking inside it. The internal states had sorted themselves into groups, and the groups were nouns and verbs, animate and inanimate, things that can eat and things that can be eaten. Nobody had given it a single category. It arrived at them because a word's category is what its next word depends on.

The modern version of that result is more startling. Take a network trained only to predict legal moves in Othello (a board game where you flip your opponent's pieces by trapping them between two of yours), never shown a board, never told the rules. Probe its activations (train a small separate classifier to read one specific fact out of a network's internal numbers) and you find the state of the board in there. Then change that internal board and watch the moves it goes on to make change with it. That is what makes it a finding rather than a curiosity. The board is not sitting in the numbers by accident. It is what the network is using.

The rule underneath both is worth saying plainly. Prediction does not reward description. It rewards whatever makes the next thing less surprising, and sometimes the cheapest way to be less surprised is to build the thing that is actually generating the data.

Sometimes. Not always, and not by magic. A predictor will happily settle for a shortcut that works on the data it saw, and much of the rest of this course is about telling the two apart.

Predict the next what?

The loop says predict the next thing. It never says what counts as a thing, and that choice decides what you end up with.

Predict the next pixel and you get a system that is very good at leaves. Leaves are genuinely hard to predict, and they are also most of the pixels, so that is where the capacity goes (*capacity* is how much a model can hold. Spend it on one thing and it is not there for another). Predict the next word and you get grammar, and a surprising amount of knowledge about the world, because that is what the next word turns on. Predict the next state of some compact description and you get something a planner can use.

Same loop, three different machines. The only thing that changed is what was put on the right-hand side of the guess.

So *predict the next thing* is not one method. It is a family, and picking the target is the design decision that matters most.

Being right is the same as being brief

There is a second way to look at the whole business, and it turns out to be the same way.

Suppose you have to send a message down a wire, and you are charged by the symbol. You and the receiver share a predictor. Before each symbol, both of you run it and get a list of what is likely next. Now you only need to send enough to pick out the right one from that list.

If the predictor was confident and correct, that costs almost nothing. If it was confident and wrong, it costs a lot. Shannon made this exact: a symbol you gave a probability of one in two costs one bit (one bit is one yes or no answer, so eight bits can pick one option out of 256), one in four costs two, one in a thousand costs about ten.

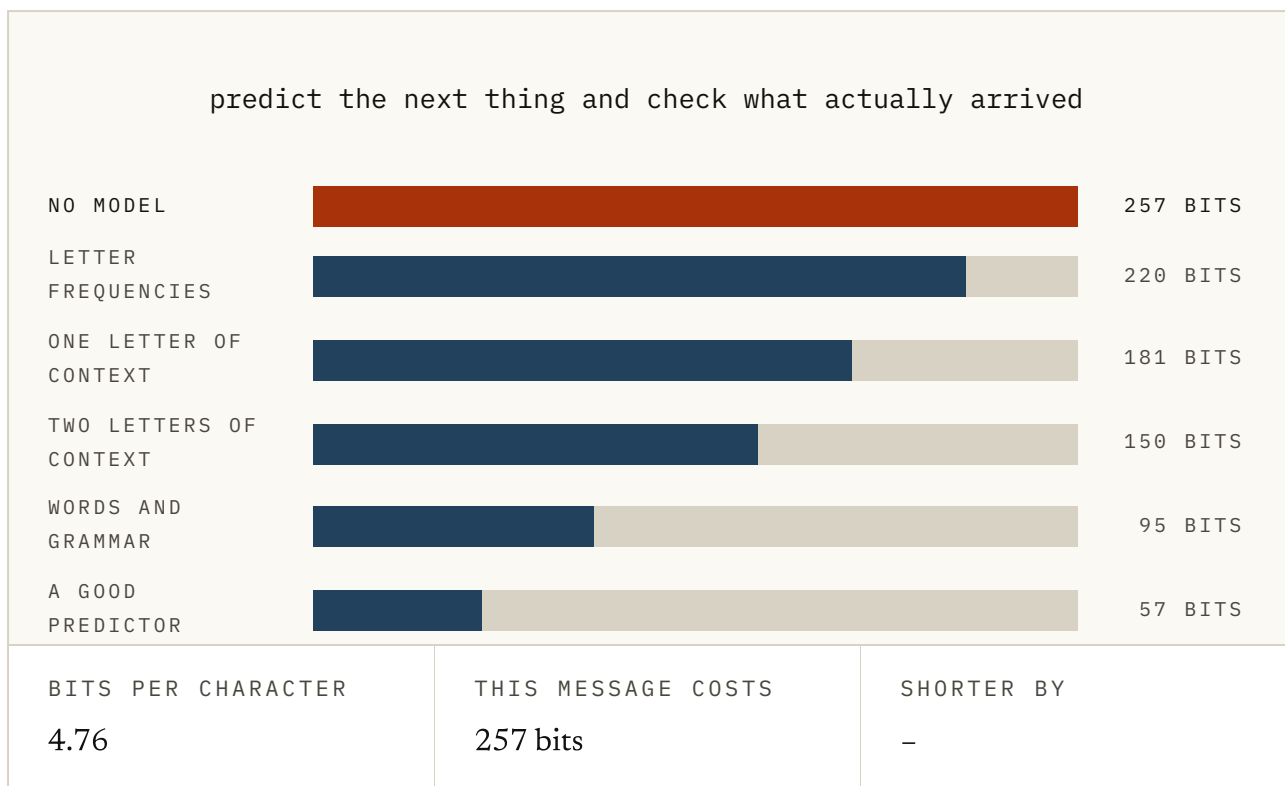


FIG. 3.3 The same sentence, written down by predictors of increasing quality. The bits per character are published estimates for English rather than anything this figure computed. The bottom row is roughly where Shannon put a person guessing English letter by letter, and where good language models sit today.

So the length of the shortest message you can write is a measurement of how well you predicted. A better predictor is a better compressor, and the same thing read backwards: whatever compresses your data the hardest has understood the most about it.

That is not a metaphor. There is a long-running prize for compressing a snapshot of Wikipedia, on the argument that you cannot squeeze text much further without knowing what the text means. Every winner so far has been, in effect, a language model.

And it explains why prediction is worth so much more than it looks. You are not asking a system to memorise. Memorising is the expensive option: a lookup table of everything that ever happened is enormous and useless on anything new. The cheap option, the one that actually gets you a shorter message, is to work out the rule that produced the data and keep that instead.

Where it stops being enough

Two honest limits, because the argument so far has been suspiciously tidy.

Predicting well is not the same as being useful. A system can be excellent at continuing a recording and still hand you nothing you can act on. Saying what comes next is not the same as saying what would come next if you did something different. Only the second one supports a decision.

Being unsurprised is not the same as being right. A predictor that has only ever seen calm weather is not surprised by calm weather, and that tells you nothing about what it would do in a storm. Low error on what you happened to record is a weaker claim than it sounds.

None of that undoes the chapter. Prediction really is enough to pull structure out of raw experience, and it really is the cheapest signal anyone has found. It is just not free, and it does not aim itself.

Try it

1 One photograph of a ball in flight. What can you not work out from it?

- a. Where the ball is
- b. How big the ball is
- c. Which way it is going and how fast
- d. What colour it is

ANSWER c. Which way it is going and how fast. Direction and speed are not in any single frame. They exist in the relationship between frames, which is exactly the kind of thing a predictor has to build for itself.

2 Why is next-thing prediction so much cheaper to train on than labelled data?

- a. The models are smaller
- b. The answer is already in the recording, so nobody has to write labels
- c. It needs less computing time
- d. It converges in fewer steps

ANSWER b. The answer is already in the recording, so nobody has to write labels. Every moment is the answer to the moment before. That turns any recording of anything into a training set, and removes the step where a person has to annotate a million examples.

3 Elman's network was trained only to predict the next word, and its internal states sorted themselves into nouns, verbs, animate and inanimate. Why?

- a. Those categories were in the training labels
- b. The architecture had one unit per category
- c. A word's category is what its next word depends on, so the categories were the cheapest way to be less wrong
- d. It memorised the corpus

ANSWER c. A word's category is what its next word depends on, so the categories were the cheapest way to be less wrong. Nothing supplied the categories. Prediction rewards whatever makes the next thing less surprising, and grammatical category happens to be exactly that.

4 A probe finds board state inside a network trained only on legal Othello moves. What turns that from a curiosity into a finding?

- a. The probe is very accurate
- b. The network was large
- c. Changing that internal board changes the moves the network then makes
- d. The board can be drawn as a picture

ANSWER c. Changing that internal board changes the moves the network then makes. Accuracy alone could be a coincidence in the numbers. Intervening on the representation and watching behaviour follow is what shows the network is actually using it.

5 Under a good predictor, a symbol you were confident about and got right costs almost nothing to send. Why?

- a. It can be left out of the message
- b. The cost of a symbol is set by the probability you gave it
- c. Common symbols are stored in a table
- d. The receiver guesses it and does not need the message

ANSWER b. The cost of a symbol is set by the probability you gave it. Shannon made the price exact. High probability means a short code, which is why a better predictor is literally a better compressor.

6 What does the compression view say about memorising the training data?

- a. It is the best available strategy
- b. It is the expensive option: a lookup table of everything is enormous and useless on anything new
- c. It compresses better than any rule
- d. It is what all learning does

ANSWER b. It is the expensive option: a lookup table of everything is enormous and useless on anything new. The short message comes from finding the rule that generated the data and keeping that instead. Memorising is what compression penalises.

7 Same loop, different target: predict the next pixel, or the next word, or the next compact state. What does that choice change?

- a. Nothing, the loop is what matters
- b. Only how long training takes
- c. What the system ends up good at, and what its capacity gets spent on
- d. Whether the method counts as self-supervised

ANSWER c. What the system ends up good at, and what its capacity gets spent on. Predict every pixel and most of the capacity goes to leaves, because that is where most of the pixels are. Picking the target is the design decision that matters most.

8 A model has very low error predicting the next frame of a recording. What does that alone not tell you?

- a. That it saw the recording
- b. What it would predict if you acted differently, and how it behaves outside what it recorded
- c. That its error was measured correctly
- d. That the recording was long enough

ANSWER b. What it would predict if you acted differently, and how it behaves outside what it recorded. Being unsurprised by what you happened to record is a weaker claim than it sounds, and saying what comes next is not the same as saying what would come next under a different action.

FIG. 3.4 Eight questions on the loop and what it produces. The last two are the ones that separate a good predictor from a useful one.

Everything here has quietly assumed the thing being predicted is small. Two numbers for a ball, one word at a time for language. A camera hands you two million numbers, thirty times a second, and almost all of them are the leaves.

Thanks for reading. Chapter 4 is about the squeeze in the middle. How a system gets from what the world looks like to what state it is in, and why that narrow point does more work than the prediction on either side of it.

Sources

A Mathematical Theory of Communication SHANNON, 1948 ↗

Where the price of a symbol becomes the probability you gave it. Everything about prediction and compression being one job starts here.

<https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>

Prediction and Entropy of Printed English SHANNON, 1951 ↗

Shannon sat people down and had them guess English one letter at a time. The bottom row of Figure 3.3 is roughly what he measured.

<https://doi.org/10.1002/j.1538-7305.1951.tb01366.x>

Finding Structure in Time ELMAN, 1990 ↗

Train a small network to predict the next word, then look inside: nouns, verbs, animate and inanimate, none of it asked for.

https://doi.org/10.1207/s15516709cog1402_1

Emergent World Representations LI ET AL., 2022 ↗

A network given nothing but legal Othello moves, with the board found inside it and causally manipulated.

<https://arxiv.org/abs/2210.13382>

Othello-GPT has a linear emergent world representation NANDA ET AL., 2023 ↗

The follow-up that sharpened what the probe was actually reading.

<https://arxiv.org/abs/2309.00941>

Language Models are Unsupervised Multitask Learners RADFORD ET AL., 2019 ↗

Next-word prediction, scaled, turning into capabilities nobody trained for. The strongest evidence that the target choice is the design decision.

https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

The Hutter Prize HUTTER, ONGOING ↗

A cash prize for compressing a snapshot of Wikipedia, run on the argument that you cannot squeeze text further without modelling what it means.

<http://prize.hutter1.net/>

Representation Learning: A Review and New Perspectives

BENGIO, COURVILLE & VINCENT, 2013 ↗

The survey that laid out what a good learned representation is for, written before prediction had finished winning the argument.

<https://arxiv.org/abs/1206.5538>

Formal theory of creativity, fun, and intrinsic motivation SCHMIDHUBER, 2010 ↗

Compression progress as a drive in its own right: not just a way to measure a model, but a reason to go and look at something.

<https://people.idsia.ch/~juergen/creativity.html>

FIG. 3.5 Ordered for someone building on this rather than for historical completeness.